A

SOFTWARE ENGINEERING CASE STUDY REPORT

ON

ONLINE FOOD DELIVERY SYSTEM (FOODPULSE)

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

BY

K. VAMSHI CHANDRA REDDY (160124737111)
B.E. V Semester, IT - 2

UNDER THE GUIDANCE OF

Ms. R. K. SWATHI,
Course Faculty, Software Engineering,
DEPT. OF IT, CBIT.



DEPARTMENT OF INFORMATION TECHNOLOGY
CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)
(Affiliated to Osmania University; Accredited by NBA and NAAC, ISO 9001:2015 Certified Institution), GANDIPET, HYDERABAD – 500 075
Website: www.cbit.ac.in

2026-2027

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
An Autonomous Institute | Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in
Approved by | Affiliated to | UGC Autonomous | 12 Programs Accredited by NBA | Grade A++ in NAAC | All India Ranking 151-200 Band nirf | Certified by ISO 21001 : 2018
COMMITTED TO RESEARCH, INNOVATION AND EDUCATION 47 years

CERTIFICATE

This is to certify that the case study report entitled "**ONLINE FOOD DELIVERY SYSTEM (FOODPULSE)**" submitted by **K. VAMSHI CHANDRA REDDY (160124737111)** in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF ENGINEERING** in **INFORMATION TECHNOLOGY** to **CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)**, affiliated to **OSMANIA UNIVERSITY**, Hyderabad, is a record of bonafide work carried out by him under my supervision and guidance in the course Software Engineering. The results embodied in this report have not been submitted to any other University for the award of any other Degree or Diploma.

Course Faculty
Ms. R. K. SWATHI
Dept. of IT,
CBIT, Hyderabad.

Head of the Department
Prof. M. Venugopalachari
Professor, Dept. of IT,
CBIT, Hyderabad.

DECLARATION

I declare that the case study report entitled "**ONLINE FOOD DELIVERY SYSTEM (FOODPULSE)**" is being submitted by me in the Department of Information Technology, Chaitanya Bharathi Institute of Technology (A), Osmania University.

This is a record of bonafide work carried out by me under the guidance and supervision of Ms. R. K. SWATHI, Dept. of IT, C.B.I.T.

No part of the work is copied from books/journals/internet and wherever the portion is taken, the same has been duly referred to in the text. The reported results are based on the project work done entirely by me and not copied from any other source.

K. VAMSHI CHANDRA REDDY
(160124737111)

ACKNOWLEDGEMENTS

I would like to express my heartfelt gratitude to Ms. R. K. SWATHI, our Software Engineering faculty, for her invaluable guidance, constant support and encouragement throughout this case study.

I am grateful to our Head of Department, Prof. M. Venugopalachari, for his steady support and for the provision of every resource required for the completion of this work.

I would like to take this opportunity to thank our Principal, Prof. C. V. Narasimhulu, as well as the Management of the Institute, for having designed an excellent learning atmosphere.

My thanks are due to all members of the staff and our lab assistants for providing me with the help required to carry out the groundwork of this project.

K. VAMSHI CHANDRA REDDY
(160124737111)

ABSTRACT

Online food delivery platforms coordinate customers, restaurants and delivery partners in real time. This Software Engineering case study documents the analysis, design, implementation and testing of FoodPulse, a multi-role Online Food Delivery System built as a web application. Customers can browse restaurants, customise dishes with add-ons, apply coupons, pay and track an order live; merchants can accept orders, progress them through the kitchen, manage menu availability and view analytics; and delivery partners can accept dispatches and mark deliveries complete.

The report follows the Software Engineering syllabus. Requirements are written as an IEEE 830 style SRS with functional and non-functional requirements and user stories. The system is modelled with use case, class, sequence and state diagrams, Level 0 and Level 1 Data Flow Diagrams and an Entity–Relationship diagram. Agile Scrum is justified as the process model; size and effort are estimated with Function Points (272 FP) and Basic COCOMO (105.4 person-months for 24 KLOC in the semi-detached mode); and a risk register records mitigation plans.

The application is implemented with React 19, TypeScript, Vite and Tailwind CSS on the front end and Supabase (PostgreSQL, Auth and Realtime) with Row Level Security on the back end. A billing engine computes delivery fee, platform fee, 5% GST and coupon discounts, and an in-app SE Studio module demonstrates the syllabus topics on the same product. Black-box, white-box and functional tests confirm the behaviour of the billing, coupon and order workflows, and screenshots of every portal are presented. Limitations, mainly the simulated payment and live map, and the future scope are also discussed.

Keywords: Software Engineering, Food Delivery, SRS, UML, DFD, Agile Scrum, COCOMO, React, Supabase.

TABLE OF CONTENTS

Abstract **5**
List of Figures **8**
List of Tables **9**
List of Abbreviations **10**
1. INTRODUCTION **11**
1.1 Overview 11
1.2 Applications 11
1.3 Motivation 11
1.4 Problem Statement 12
1.5 Objectives 12
1.6 Scope and Organisation of the Report 12
2. LITERATURE SURVEY **13**
2.1 Existing Food Delivery Platforms 13
2.2 Process Models Reviewed 13
2.3 Web Technologies and Backend-as-a-Service 14
2.4 Techniques for Real-Time Order Tracking 14
2.5 Gap Analysis 14
3. SYSTEM REQUIREMENT SPECIFICATION **15**
3.0 Purpose, Scope and User Classes 15
3.1 Functional Requirements 15
3.2 Non-Functional Requirements 16
3.3 Software Requirements 16
3.4 Hardware Requirements 17
3.5 User Stories and Acceptance Criteria 17
4. PROPOSED METHODOLOGY **18**
4.1 Process Model: Agile Scrum 18
4.2 Feasibility Study 18
4.3 Project Estimation 18
4.4 Risk Analysis 19
4.5 System Architecture 20
4.6 Use Case Model 20
4.7 Data Flow Diagrams 21
4.8 Class Diagram 23
4.9 Sequence Diagram 23
4.10 Order State Model 24

4.11 Database Design..... 24
5. SYSTEM IMPLEMENTATION..... 26
5.1 Technology Stack and Project Structure..... 26
5.2 Authentication and Role-Based Routing..... 26
5.3 Customer Portal, Dish Modal and Cart..... 26
5.4 Billing and Coupon Engine..... 26
5.5 Order Service and Persistence..... 27
5.6 Live Order Tracker..... 27
5.7 Merchant Dashboard and Driver Portal..... 27
5.8 Security: Row Level Security Policies..... 28
5.9 SE Studio Module..... 28
5.10 Deployment..... 28
6. TESTING AND RESULTS..... 29
6.1 Test Strategy..... 29
6.2 Black-Box Test Cases..... 29
6.3 White-Box Test Cases..... 29
6.4 Functional (UI) Test Cases..... 30
6.5 Quality Attributes..... 30
6.6 Results: Application Screenshots..... 31
6.7 Discussion of Results..... 34
7. CONCLUSION AND FUTURE SCOPE..... 35
7.1 Conclusion..... 35
7.2 Limitations..... 35
7.3 Future Scope..... 35
BIBLIOGRAPHY..... 36
APPENDIX..... 36
Appendix A: Database Schema Excerpt (supabase/schema.sql)..... 36
Appendix B: Project Links..... 37

LIST OF FIGURES

Figure 4.1: Basic COCOMO effort for 24 KLOC by project mode.....19
Figure 4.2: Layered architecture of FoodPulse.....20
Figure 4.3: Use case diagram.....21
Figure 4.4: Level 0 DFD (context diagram).....22
Figure 4.5: Level 1 DFD.....22
Figure 4.6: Class diagram.....23
Figure 4.7: Sequence diagram: placing and fulfilling an order.....24
Figure 4.8: State diagram of an order.....24
Figure 4.9: Entity–relationship diagram.....25
Figure 6.1: Authentication portal with role cards and demo login.....31
Figure 6.2: Customer home with restaurants, categories and search.....32
Figure 6.3: Dish modal with size and add-on selection.....32
Figure 6.4: Cart drawer with coupon SE2026 applied and bill breakdown.....32
Figure 6.5: Live order tracker with progress stages and map markers.....33
Figure 6.6: My Orders with an active and a delivered order.....33
Figure 6.7: Merchant dashboard with live kitchen orders.....33
Figure 6.8: Driver portal with active delivery mission.....34
Figure 6.9: SE Studio module showing process models.....34

LIST OF TABLES

Table 2.1: Process models reviewed for FoodPulse 13
Table 2.2: Comparison of typical commercial apps and FoodPulse 14
Table 3.1: User classes and their characteristics 15
Table 3.2: Functional requirements 15
Table 3.3: Non-functional requirements 16
Table 3.4: Software requirements 16
Table 3.5: Hardware requirements 17
Table 3.6: Selected user stories 17
Table 4.1: Sprint plan 18
Table 4.2: Unadjusted function point calculation 18
Table 4.3: Risk register 19
Table 4.4: Use case specification: Place Order 21
Table 4.5: Main database tables 25
Table 5.1: Coupon rules 27
Table 5.2: Row Level Security summary 28
Table 6.1: Black-box test cases for billing and coupons 29
Table 6.2: White-box test cases 29
Table 6.3: Functional test cases per role 30
Table 6.4: Quality attribute assessment 30

LIST OF ABBREVIATIONS

API:	Application Programming Interface
BaaS:	Backend-as-a-Service
COCOMO:	Constructive Cost Model
COD:	Cash on Delivery
CSS:	Cascading Style Sheets
DFD:	Data Flow Diagram
ER:	Entity–Relationship
FP:	Function Point
FR / NFR:	Functional / Non-Functional Requirement
FURPS:	Functionality, Usability, Reliability, Performance, Supportability
GST:	Goods and Services Tax
GPS:	Global Positioning System
IEEE:	Institute of Electrical and Electronics Engineers
JWT:	JSON Web Token
KLOC:	Thousand Lines of Code
RLS:	Row Level Security
SDLC:	Software Development Life Cycle
SQL:	Structured Query Language
SRS:	Software Requirements Specification
UI:	User Interface
UML:	Unified Modeling Language
UPI:	Unified Payments Interface
UUID:	Universally Unique Identifier

CHAPTER 1
INTRODUCTION

1.1 Overview

Online food delivery has become one of the most visible examples of software that links many independent parties in real time. A single order involves a hungry customer, a restaurant kitchen, a delivery partner on a motorbike, a payment network and a platform that coordinates all of them. If any one link is slow, wrong or unclear, the customer ends up with cold food, the restaurant loses a sale and the driver loses time. Building such a system is therefore not just a programming exercise; it is a software engineering problem that needs a clear requirements specification, a sound design, a disciplined process, estimation, risk control and thorough testing.

This report presents a Software Engineering case study of **FoodPulse**, a multi-role Online Food Delivery System developed as a web application. FoodPulse lets customers browse restaurants, customise dishes with add-ons, apply promotional coupons, pay and track the delivery live on a map. It lets restaurant owners (merchants) receive and progress orders, manage their menu and view analytics, and it lets delivery partners (drivers) accept dispatches and mark orders as delivered. The application is built with React 19, TypeScript, Vite and Tailwind CSS on the front end and uses Supabase (PostgreSQL, authentication and realtime channels) as the back end. The source code is available at github.com/vamshi1010-cr/food-pulse-delivery and a deployed build is hosted on Vercel.

Apart from the working application, the project includes an in-app “SE Studio” module that demonstrates the topics of the Software Engineering syllabus on the same product: process models, SRS, COCOMO and Function Point estimation, risk analysis, DFDs, test cases and software quality metrics. This report follows the same structure and documents how each syllabus concept was applied to FoodPulse.

1.2 Applications

A system of this kind has applications well beyond a single restaurant ordering page:

- **Multi-restaurant marketplaces** where several independent kitchens list menus on one platform and share a common delivery fleet.
- **Cloud kitchens and cafeterias**, including campus canteens, where pre-ordering reduces queues and food wastage.
- **Hyper-local delivery** of groceries, bakery items and medicines, because the same order–dispatch–track workflow applies.
- **Analytics for restaurant owners**, who can see order volume, revenue and best-selling dishes from the dashboard.

1.3 Motivation

Food delivery apps are used by almost every student and working professional, yet few students see how much engineering lies behind one order. We chose this system because it touches every unit of the Software Engineering course in a realistic way. It has several user classes with conflicting needs, a non-trivial billing rule set (delivery fee, platform fee, GST and coupons), a state machine for the order life cycle, a real-time

component and a data model with security rules. Documenting these aspects with proper SRS, UML and DFD artefacts and then verifying them against a working implementation gave us a complete view of the software life cycle rather than a set of disconnected diagrams.

1.4 Problem Statement

Small and medium restaurants often depend on phone calls or on third-party aggregators that charge high commissions, while customers face unclear prices, no reliable delivery tracking and fragmented apps for ordering and support. The problem is to **design and develop a web-based Online Food Delivery System that connects customers, restaurants and delivery partners on one platform, calculates the bill transparently, keeps all parties informed of the order status in real time, and is built and documented using established Software Engineering practices.**

1.5 Objectives

1. To elicit and document functional and non-functional requirements in an IEEE 830 style SRS for the three user roles.
2. To model the system using UML (use case, class, sequence and state diagrams), Data Flow Diagrams (Level 0 and Level 1) and an Entity–Relationship diagram.
3. To select a suitable process model and justify it, and to estimate size, effort, time and cost with Function Points and COCOMO.
4. To identify and plan mitigation for project risks.
5. To implement a responsive, role-based web application with cart, coupon, billing, order tracking, merchant and driver workflows.
6. To design a secure relational database using Row Level Security so that every role sees only the data it should.
7. To verify the system with black-box and white-box test cases and report the results with screenshots.

1.6 Scope and Organisation of the Report

The scope covers the customer, merchant and driver portals, authentication, the billing engine, order life cycle and tracking. Payment and the live map are simulated at the user-interface level (see Chapter 7). Chapter 2 reviews existing systems, Chapter 3 gives the SRS, Chapter 4 the methodology, estimation and design, Chapter 5 the implementation, Chapter 6 testing and results, and Chapter 7 the conclusion and future scope.

CHAPTER 2
LITERATURE SURVEY

Before designing FoodPulse we studied existing delivery platforms, the process models discussed in the Software Engineering course, and the technologies available for building a real-time web application. This chapter summarises that study and identifies the gap that the project addresses.

2.1 Existing Food Delivery Platforms

Zomato and Swiggy are the best-known Indian platforms. Both list thousands of restaurants, offer coupon-based discounts, show a live delivery map and charge a mix of delivery, platform and tax components on every bill. Their bills therefore contain a structure very similar to the one in FoodPulse: item total, delivery fee, platform fee, GST and discount. Restaurants use a separate partner application to accept orders and update availability, and drivers use a third application for dispatch.

Uber Eats and DoorDash popularised the three-sided marketplace model in which customers, merchants and couriers each have their own interface, and the platform matches couriers to orders. Their design shows how an order moves through a fixed sequence of states and how every state change must be visible to all three parties.

Our observation is that these commercial systems solve the problem at very large scale, but their internal design is not available to students. A small, fully documented reference implementation that shows the engineering decisions behind such a system is useful for learning, and that is the role of FoodPulse.

2.2 Process Models Reviewed

Sommerville and Pressman describe several software process models. We reviewed the five models implemented in the SE Studio module and scored their suitability for a food delivery project (Table 2.1). The scores reflect how well each model handles changing requirements, frequent releases and continuous user feedback, which are typical of consumer applications.

Table 2.1: Process models reviewed for FoodPulse

Model	Category	Strength	Weakness for this project
Waterfall	Prescriptive	Clear phases and documents	Cannot absorb requirement changes late in the cycle
Prototyping	Evolutionary	Early UI feedback from users	Risk of throw-away code and poor structure
RAD	Specialised	Very fast delivery with components	Needs modular design and strong team
Spiral	Evolutionary	Explicit risk analysis in each loop	Heavy management overhead for a small team
Agile Scrum	Agile	Short sprints, working software, adapts to change	Needs discipline in documentation

Agile Scrum was selected because the three user portals could be delivered incrementally in sprints, and each sprint produced a demonstrable increment that could be reviewed with the guide.

2.3 Web Technologies and Backend-as-a-Service

React popularised component-based user interfaces in which the screen is a function of state, which suits dashboards where the order list changes constantly. TypeScript adds static types, which helped us keep the data structures (Order, CartItem, MenuItem) consistent across components and services. Vite provides fast builds, and Tailwind CSS allows a consistent dark and light theme to be implemented with utility classes.

For the back end, a conventional approach would be a Node.js server with REST APIs and a separate database. A Backend-as-a-Service such as Supabase instead provides a managed PostgreSQL database, JWT-based authentication and a realtime channel through one client library. Because PostgreSQL supports Row Level Security (RLS), authorisation rules can be written next to the data, which is attractive for a multi-role application.

2.4 Techniques for Real-Time Order Tracking

Three techniques are commonly used to keep a customer informed: (a) periodic polling of the server, which is simple but wasteful; (b) WebSockets, which keep a persistent connection and push updates; and (c) database change feeds, in which the database publishes row changes to subscribed clients. FoodPulse adopts the third approach through Supabase Realtime on the orders table, and its live tracker view renders the driver, restaurant and customer positions on a map-style canvas.

2.5 Gap Analysis

Table 2.2: Comparison of typical commercial apps and FoodPulse

Aspect	Commercial apps	FoodPulse
Source / design visibility	Closed	Open code with SRS, UML and DFD
Roles in one product	Separate apps per role	Customer, merchant and driver in one web app
Bill transparency	Itemised bill	Itemised bill with coupon rule messages
Authorisation	Server side, hidden	Documented RLS policies per role
Software engineering evidence	Not available	Estimation, risks, tests and quality metrics in SE Studio
Deployment	Large cloud infrastructure	Static front end on Vercel, managed Supabase

The gap that FoodPulse fills is therefore an educational and practical one: a compact, well-documented, multi-role system whose requirements, design and tests are traceable to a running implementation.

CHAPTER 3
SYSTEM REQUIREMENT SPECIFICATION

This chapter follows the structure of IEEE Std 830-1998 for a Software Requirements Specification. It records what the system must do (functional requirements), how well it must do it (non-functional requirements) and the environment in which it runs.

3.0 Purpose, Scope and User Classes

The purpose of FoodPulse is to provide a single platform on which food can be ordered from partner restaurants and delivered by independent drivers. The product is a cross-browser, mobile-responsive web application. Four user classes are identified in Table 3.1.

Table 3.1: User classes and their characteristics

User class	Description	Main goals
Customer	Hungry consumer using a phone or laptop; low technical skill	Find food quickly, understand the price, know when it will arrive
Merchant	Restaurant owner or manager using a tablet or laptop in the kitchen	Receive orders, update status, keep the menu accurate, see sales
Delivery partner	Driver using a phone while riding	See nearby jobs, accept one, finish it with minimal taps
Administrator	Maintainer of the platform (through Supabase console)	Manage data, coupons and access

3.1 Functional Requirements

The functional requirements are grouped by module. Each requirement has an identifier so that test cases in Chapter 6 can refer to it.

Table 3.2: Functional requirements

ID	Module	Requirement
FR-01	Authentication	Users shall register and sign in with email and password and choose a role (customer, merchant, driver). A demo login shall be available for evaluation.
FR-02	Session	The system shall restore the previous session on page reload and shall allow sign-out, which clears the cart and order state.
FR-03	Browse	Customers shall browse restaurants and filter them by search text, category, pure-veg and minimum rating.
FR-04	Dish options	Customers shall open a dish, select a size, add-ons and write cooking instructions; the dish price shall update as options change.
FR-05	Cart	Customers shall add, remove and change the quantity of items in a cart drawer.
FR-06	Billing	The system shall compute subtotal, delivery fee, platform fee, 5% GST and discount and display the amount payable.
FR-07	Coupons	The system shall validate the coupons SE2026, CBITFOOD and FIRST50 against their rules and show a clear error for invalid or ineligible codes.
FR-08	Checkout	Customers shall select a delivery address and a payment method (UPI, card, wallet, cash on delivery) and place the order.

ID	Module	Requirement
FR-09	Tracking	Customers shall view a live tracker with the order stages and the positions of driver, restaurant and delivery address.
FR-10	History	Customers shall view past orders and reorder the same dishes.
FR-11	Merchant orders	Merchants shall see incoming orders and move them through Confirmed, Preparing and Out-for-delivery (handover to driver).
FR-12	Menu manager	Merchants shall toggle the availability of a dish and view analytics of orders and revenue.
FR-13	Dispatch	Drivers shall see orders available for pickup and accept one.
FR-14	Delivery	Drivers shall see pickup and drop-off details, share location and mark the order delivered.
FR-15	Theme	Users shall switch between dark and light themes; the choice shall persist across visits.

3.2 Non-Functional Requirements

Table 3.3: Non-functional requirements

ID	Attribute	Requirement
NFR-01	Performance	The first screen shall load within 2 seconds on a 4G connection; interactions such as adding to cart shall respond within 100 ms.
NFR-02	Security	Passwords shall be handled only by Supabase Auth; sessions use JWT; every table shall have Row Level Security enabled.
NFR-03	Usability	A new customer shall be able to place an order in at most six steps without training; layouts shall be responsive from 360 px upwards.
NFR-04	Reliability	If the back end is not configured or unreachable, the application shall continue in a demo mode with local mock data instead of crashing.
NFR-05	Availability	The front end is static and served from a CDN; the target availability follows the hosting provider’s service level.
NFR-06	Maintainability	Code shall be written in TypeScript with typed interfaces, and separated into components, services and data modules.
NFR-07	Portability	The application shall run in current versions of Chrome, Edge, Firefox and Safari on desktop and mobile.
NFR-08	Accessibility	Colour contrast shall be adequate in both themes and controls shall be reachable by keyboard.

3.3 Software Requirements

Table 3.4: Software requirements

Component	Specification
Operating system	Windows 10/11, Linux or macOS for development; any OS with a modern browser for use
Runtime and tools	Node.js 20 or later, npm, Git
Front end	React 19, TypeScript, Vite, Tailwind CSS 4, lucide-react, canvas-confetti
Back end	Supabase (PostgreSQL, Auth, Realtime) using @supabase/supabase-js
Hosting	Vercel for the front end
Design and documentation	StarUML / Graphviz for diagrams, LaTeX or Word for documents

Component	Specification
Browser	Latest Chrome, Edge, Firefox or Safari

3.4 Hardware Requirements

Table 3.5: Hardware requirements

Item	Developer machine	End user device
Processor	Dual-core 2.0 GHz or better	Any device with a modern browser
Memory	8 GB RAM recommended	2 GB RAM
Storage	1 GB free (node modules and build)	Negligible
Network	Broadband for npm and Supabase	4G / Wi-Fi
Display	1366 × 768 or above	360 px width or above

3.5 User Stories and Acceptance Criteria

Because the project follows Scrum, requirements were also captured as user stories with acceptance criteria (Table 3.6). Story points use the Fibonacci scale and drive the sprint plan.

Table 3.6: Selected user stories

ID	Epic	Story	Points	Acceptance criterion
US-01	Ordering	As a customer I want to customise a dish so that it suits my taste	5	Selected add-ons change the price shown on the Add button
US-02	Checkout	As a customer I want to apply a coupon so that I pay less	3	Valid code reduces the bill; invalid code shows an error
US-03	Tracking	As a customer I want to see my driver on a map so that I know when food arrives	8	Tracker shows driver, restaurant and home markers and five stages
US-04	Restaurant	As a merchant I want to mark a dish unavailable so that customers cannot order it	2	Dish disappears from ordering when toggled off
US-05	Delivery	As a driver I want to accept a nearby order so that I can earn	5	Accepted order moves to Out for delivery and shows pickup and drop details

CHAPTER 4
PROPOSED METHODOLOGY

4.1 Process Model: Agile Scrum

The development followed Agile Scrum with four one-week sprints. The product backlog was built from the user stories in Section 3.5. Each sprint ended with a review of a working increment and a short retrospective.

Table 4.1: Sprint plan

Sprint	Goal	Increment delivered
1	Foundation	Project setup, data model, authentication, role-based routing, theme
2	Customer journey	Restaurant browsing, dish modal with add-ons, cart drawer, billing and coupons
3	Fulfilment	Order service, live tracker, merchant dashboard, driver portal, order history
4	Quality	Row Level Security, SE Studio module, testing, deployment and documentation

4.2 Feasibility Study

- **Technical feasibility:** all technologies are mature and free to use at student scale; the team already knew JavaScript and SQL.
- **Economic feasibility:** Vercel and Supabase free tiers cover development and demonstration, so the cash cost is nil. The estimated effort is discussed in Section 4.3.
- **Operational feasibility:** the three portals use familiar patterns (cards, carts, status chips), so users need no training.
- **Schedule feasibility:** four sprints fit within the semester calendar.

4.3 Project Estimation

4.3.1 Function Point Analysis

Function Points measure the functionality delivered to the user independent of technology. The counts for FoodPulse are shown in Table 4.2 with average weights.

Table 4.2: Unadjusted function point calculation

Component	Count	Weight	Points
External Inputs (EI)	12	4	48
External Outputs (EO)	15	5	75
External Inquiries (EQ)	8	4	32
Internal Logical Files (ILF)	6	10	60
External Interface Files (EIF)	4	7	28
Unadjusted FP			**243**

With a Value Adjustment Factor of 1.12 the adjusted size is FP = 243 × 1.12 = **272 FP**. Assuming about 50 lines of TypeScript per function point, the size is roughly 13.6 KLOC. The implemented front end contains about 5.2 KLOC of TypeScript/TSX, because React components and the managed back end remove a large amount of boilerplate that the 50 LOC/FP ratio assumes. We therefore used a more conservative 24 KLOC (which also covers the back end, tests and documentation tooling of a production version) for COCOMO below.

4.3.2 Basic COCOMO

Basic COCOMO estimates effort E = a(KLOC)^b person-months and development time D = c(E)^d months. For the semi-detached mode (a = 3.0, b = 1.12, c = 2.5, d = 0.35) and 24 KLOC:

E = 3.0 × 24^1.12 = **105.4 person-months**; D = 2.5 × 105.4^0.35 = **12.8 months**; average staff = E / D = **8.3 persons**; cost at ₹85,000 per person-month = **₹89.6 lakh**.

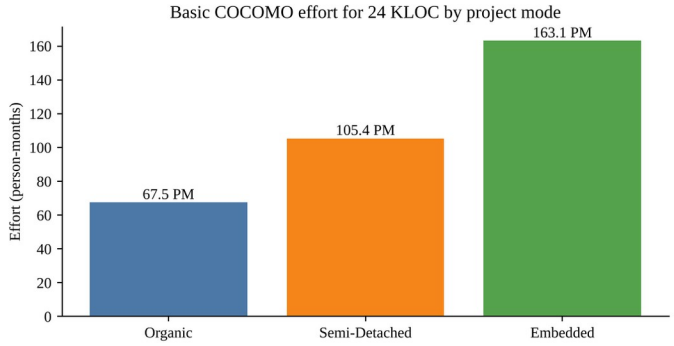


Figure 4.1: Basic COCOMO effort for 24 KLOC by project mode

The estimate describes a commercial-scale product. The student prototype was built by a much smaller team in four weeks by reusing a managed back end and open-source libraries, which illustrates how component reuse changes the effort curve.

4.4 Risk Analysis

Risks were assessed by probability and impact, and the risk score is the product of the two ratings (High = 3, Medium = 2, Low = 1 for probability; Critical = 4 … Low = 1 for impact).

Table 4.3: Risk register

ID	Risk	Prob.	Impact	Mitigation
R1	Map API rate limits or latency at lunch peaks	High	High	Cache coordinates, throttle updates to 5 s, fall back to open map tiles
R2	Payment callback timeout during UPI verification	Medium	Critical	Retry with exponential back-off, double verification
R3	Driver shortage in heavy rain	High	High	Dynamic surge pricing, wider delivery radius
R4	Unauthorised data access	Low	Critical	Row Level Security on all tables, JWT sessions
R5	Back end unavailable during demo	Medium	Medium	Demo mode with mock data (NFR-04)
R6	Requirement change late in project	Medium	Medium	Agile sprints and backlog grooming

4.5 System Architecture

FoodPulse uses a three-layer architecture (Figure 4.2). The presentation layer holds four React portals. The service layer contains authService and orderService, which wrap all calls to the back end, and a central application state held with React hooks. The data layer is Supabase: Auth for identities, PostgreSQL for data and Realtime for pushing order changes back to the clients. Keeping Supabase calls inside services means that components never contain SQL or table names, which makes the UI easy to test with mock data.

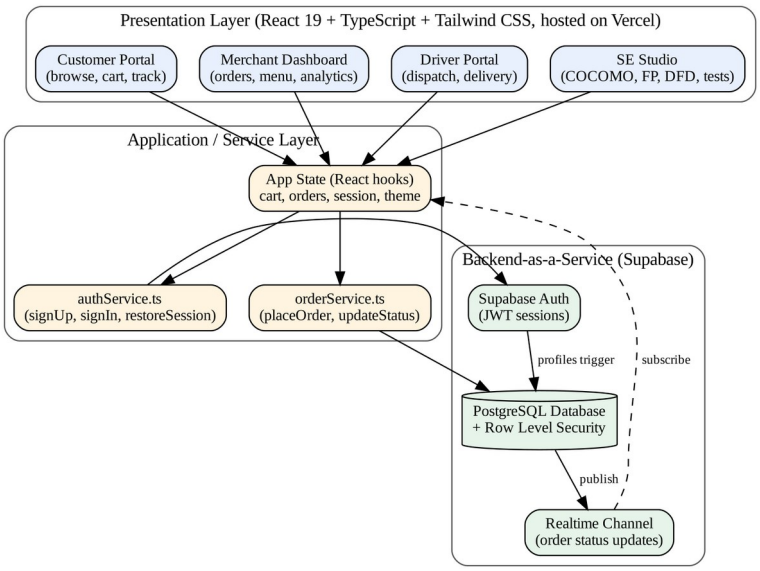


Figure 4.2: Layered architecture of FoodPulse

4.6 Use Case Model

Figure 4.3 shows the use cases of the three actors and the Payment Gateway as a secondary actor. “Customise Dish” extends “Browse & Search Restaurants” because customisation is optional, and “Place Order & Pay” always includes “Manage Cart & Apply Coupon”.



Figure 4.3: Use case diagram

Table 4.4: Use case specification: Place Order

Field	Description
Actor	Customer (primary), Payment Gateway (secondary)
Pre-condition	Customer is signed in and the cart contains at least one item
Main flow	1. Customer opens the cart. 2. System shows the itemised bill. 3. Customer optionally applies a coupon. 4. Customer selects address and payment method. 5. Customer confirms. 6. System creates the order with status PLACED and opens the tracker.
Alternate flow	3a. Coupon invalid or ineligible: system shows the rule message and the bill is unchanged.
Post-condition	Order and order items are stored; merchant sees the new order; cart is cleared

4.7 Data Flow Diagrams

The context diagram (Level 0, Figure 4.4) shows FoodPulse as a single process exchanging data with four external entities. Level 1 (Figure 4.5) decomposes it into six processes and five data stores. The flows are named so that each can be traced to a functional requirement; for example, “priced order” from process 3.0 to 4.0 corresponds to FR-06 to FR-08.

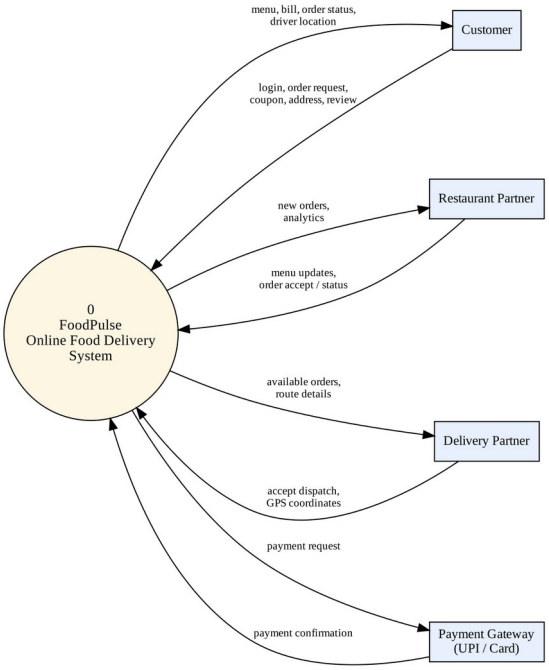


Figure 4.4: Level 0 DFD (context diagram)

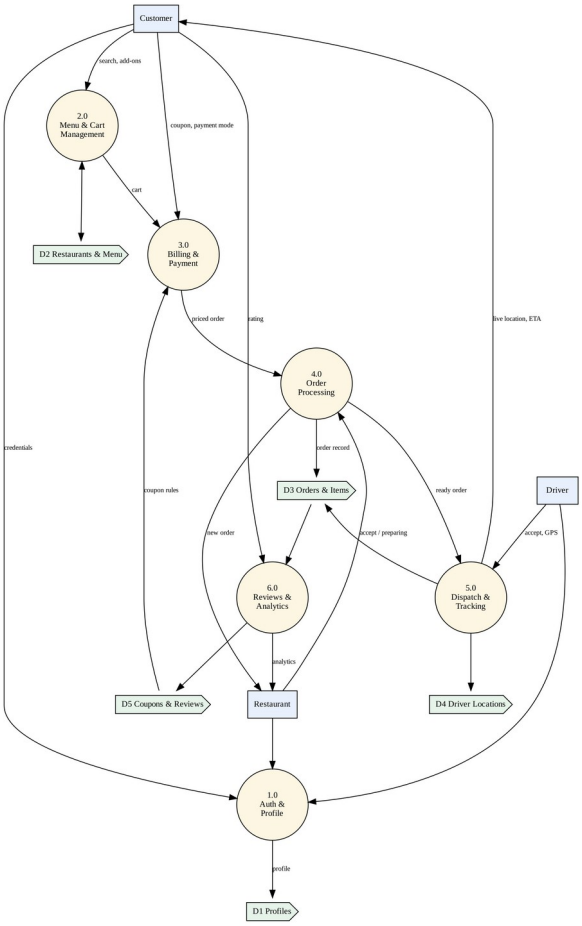


Figure 4.5: Level 1 DFD

4.8 Class Diagram

Customer, Merchant and Driver specialise an abstract User. A Merchant owns restaurants, a Restaurant offers menu items, and an Order is made of cart items that refer to menu items. An order may use one coupon and may be rated by one review, and it is delivered by at most one driver (Figure 4.6).

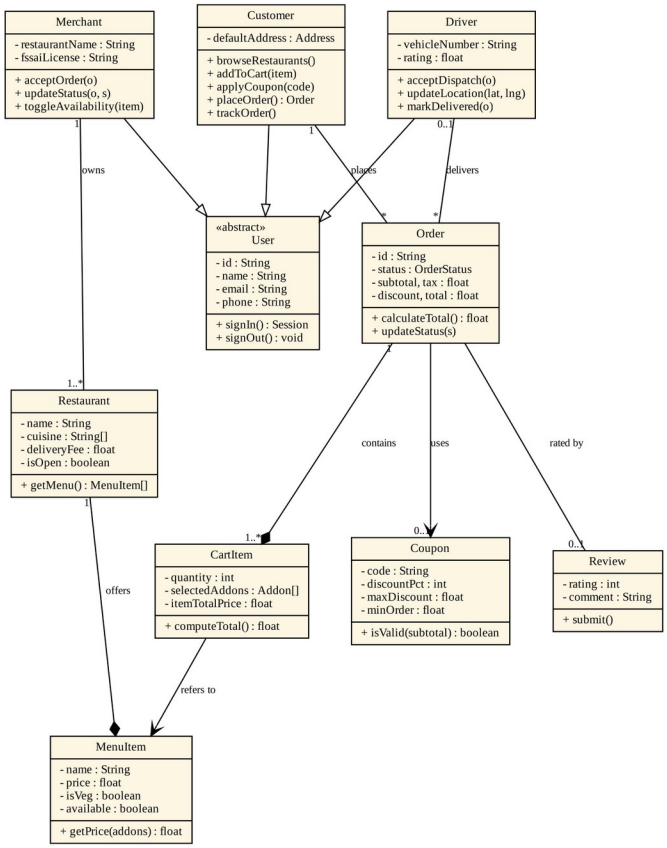


Figure 4.6: Class diagram

4.9 Sequence Diagram

Figure 4.7 shows the interactions for the most important scenario, from adding a dish to the final delivery notification. The realtime push in message 14 is what keeps the customer's tracker in step with the database.

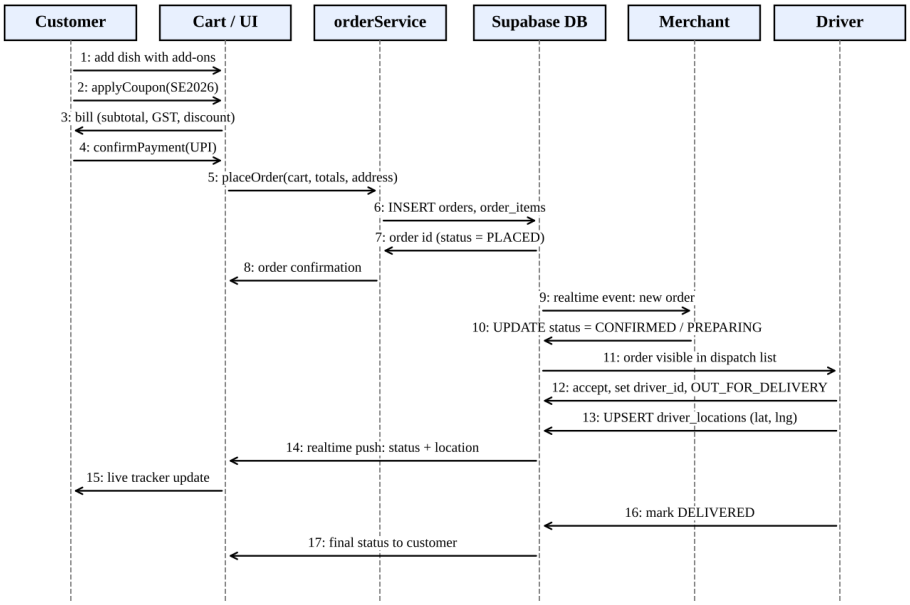


Figure 4.7: Sequence diagram: placing and fulfilling an order

4.10 Order State Model

An order moves through a fixed life cycle. The database enforces it with a CHECK constraint on the status column, and the user interface exposes only the next valid action to each role: the merchant moves PLACED → CONFIRMED → PREPARING and hands the order over (OUT_FOR_DELIVERY), which a driver can also do by accepting a dispatch, and the driver then marks it DELIVERED. The schema additionally defines READY and CANCELLED for future use; the current interface does not expose them.

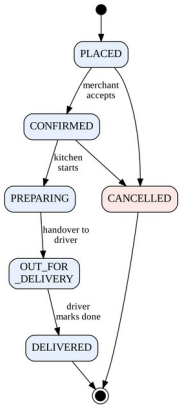


Figure 4.8: State diagram of an order

4.11 Database Design

The relational schema has eight tables (Figure 4.9). Profiles extend Supabase's auth.users table through a trigger that creates a profile row with the chosen role when a user signs up. Table 4.5 lists the main tables and their purpose.

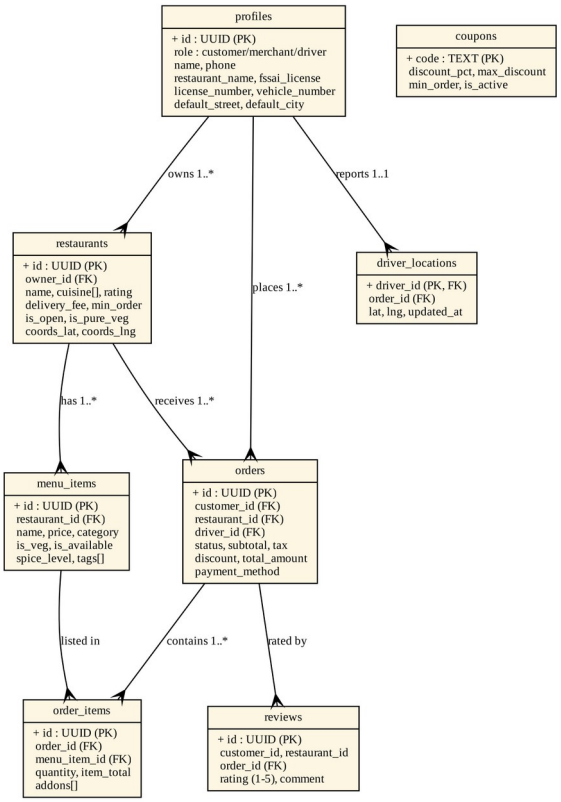


Figure 4.9: Entity–relationship diagram

Table 4.5: Main database tables

Table	Purpose	Key columns
profiles	One row per user with role and role-specific details	id (FK to auth.users), role, name, vehicle_number
restaurants	Partner restaurants	id, owner_id, cuisine[], delivery_fee, is_open
menu_items	Dishes offered	restaurant_id, price, is_veg, is_available
orders	Order header and bill breakdown	customer_id, restaurant_id, driver_id, status, total_amount
order_items	Lines of an order	order_id, menu_item_id, quantity, addons[]
reviews	Ratings with comments	order_id, rating (1–5)
driver_locations	Latest driver position for tracking	driver_id, lat, lng, updated_at
coupons	Promotional codes	code, discount_pct, max_discount, min_order

CHAPTER 5
SYSTEM IMPLEMENTATION

5.1 Technology Stack and Project Structure

The front end is a single-page React application written in TypeScript and bundled by Vite. Tailwind CSS version 4 supplies styling and lucide-react supplies the icons. The folder structure separates concerns:

```
src/
  components/
    auth/AuthPortal.tsx        login / register, demo logins
    food/Navbar, CustomerPortal, DishModal, CartDrawer,
         LiveOrderTracker, CustomerOrders,
         MerchantDashboard, DriverPortal
    se/SEStudio.tsx            software engineering studio
  services/authService.ts      signUp, signIn, signOut, restoreSession
  services/orderService.ts     placeOrder, order status updates
  lib/supabase.ts              client + isSupabaseConfigured()
  types/ (food.ts, auth.ts, se.ts)   typed domain model
  data/ (mockFoodData.ts, mockSEData.ts)   demo data
supabase/schema.sql            tables, RLS policies, seed data
```

5.2 Authentication and Role-Based Routing

AuthPortal offers three role cards (Customer, Merchant, Driver). The authService wraps supabase.auth.signUp and signInWithPassword and returns a UserSession object containing the role. On start-up App.tsx calls restoreSession(), and after a successful login it switches the active view according to the role, so a merchant never sees the customer storefront and vice versa. When Supabase environment variables are absent, isSupabaseConfigured() returns false and the demo logins create an in-memory session, which satisfies NFR-04.

5.3 Customer Portal, Dish Modal and Cart

CustomerPortal lists restaurants with cuisine chips, ratings, delivery time and discount badges, together with a search box and category filters. Selecting a dish opens DishModal, where required add-on groups (for example size) and optional groups (extra raita, boiled egg) can be chosen. Each selection produces a CartItem with its own itemTotalPrice, so the same dish with different add-ons becomes separate lines in the cart.

5.4 Billing and Coupon Engine

Billing is computed inside CartDrawer from the cart items every time the cart or coupon changes. The rules are: delivery fee ₹35, free when the subtotal reaches ₹600; platform fee ₹10; GST 5% of the subtotal rounded to the nearest rupee; and one coupon at a time. The core of the engine is shown below.

```
const subtotal    = cartItems.reduce((s, i) => s + i.itemTotalPrice, 0);
const deliveryFee = subtotal > 0 ? (subtotal >= 600 ? 0 : 35) : 0;
const platformFee = subtotal > 0 ? 10 : 0;
const tax         = Math.round(subtotal * 0.05);        // 5% GST

let discount = 0;
if (appliedCoupon === 'SE2026')        discount = Math.min(Math.round(subtotal *
0.5), 100);
```

```
else if (appliedCoupon === 'CBITFOOD') discount = Math.min(Math.round(subtotal *
0.2), 150);
else if (appliedCoupon === 'FIRST50')  discount = 50;

const totalAmount = Math.max(0, subtotal + deliveryFee + platformFee + tax -
discount);
```

Table 5.1: Coupon rules

Code	Rule	Condition
SE2026	50% off, maximum ₹100	Subtotal at least ₹250
CBITFOOD	20% off, maximum ₹150	None
FIRST50	Flat ₹50 off	None

5.5 Order Service and Persistence

When the customer confirms, placeOrder() in orderService.ts first builds a complete Order object locally, so the user interface always has something to show, and then, if Supabase is configured, inserts a row into orders followed by the lines in order_items. The function returns an object with either the order or an error string, which the caller turns into a message; exceptions never reach the user interface.

```
export async function placeOrder(customerId, cartItems, financials, address)
  : Promise<{ order: Order | null; error: string | null }> {
  const mockOrder: Order = { id: `ord-${...}`, status: 'PLACED', ... };
  if (!isSupabaseConfigured()) return { order: mockOrder, error: null };
  try {
    const { data: orderRow, error } = await supabase.from('orders').insert({
      customer_id: customerId, status: 'PLACED',
      subtotal: financials.subtotal, delivery_fee: financials.deliveryFee,
      platform_fee: financials.platformFee, tax: financials.tax,
      discount: financials.discount, total_amount:
financials.totalAmount, ... });
    ...
  } catch (e) { return { order: mockOrder, error: String(e) }; }
}
```

5.6 Live Order Tracker

LiveOrderTracker shows a five-stage progress bar (Placed, Confirmed, Preparing, Out for delivery, Delivered), the estimated arrival time and a map-style canvas with markers for the driver, the restaurant and the delivery address. While the order is out for delivery the estimated time counts down, an in-app chat with the driver is simulated, and a confetti animation (canvas-confetti) celebrates delivery. In database-backed mode orderService.subscribeToOrder() subscribes to changes on the order row through a Supabase Realtime channel.

5.7 Merchant Dashboard and Driver Portal

MerchantDashboard has three tabs: Live Kitchen Orders, Menu Manager and Analytics. Incoming orders appear as cards with their items and total, with one action button per state: “Accept & Confirm”, “Start Cooking (In Kitchen)” and “Handover to Delivery Driver”. The menu manager toggles the available flag of each dish, and the Analytics tab summarises orders and revenue.

DriverPortal shows today's earnings, a list of orders whose status is CONFIRMED or PREPARING as available dispatches, and the active delivery with pickup and drop-off cards. An "Accept Dispatch" button assigns the order and moves it to OUT_FOR_DELIVERY; a final control marks it DELIVERED.

5.8 Security: Row Level Security Policies

Every table has RLS enabled. Policies express the rules of the SRS directly in the database, so even a modified client cannot read or change data that its role is not allowed to access. Table 5.2 summarises the policies.

Table 5.2: Row Level Security summary

Table	Read	Write
profiles	Everyone (basic info)	Owner only
restaurants / menu_items	Everyone	Owning merchant only
orders	Customer: own orders; merchant: own restaurant; driver: assigned and available orders	Customer inserts; merchant and driver update status
order_items	Customer, merchant and driver of that order	Customer inserts
reviews	Everyone	Customer for own orders
driver_locations	Everyone	The driver only
coupons	Everyone	Administrator (console)

5.9 SE Studio Module

SE Studio is a sixth area of the application that demonstrates the syllabus on the FoodPulse product. It contains the process model comparison with suitability scores, an interactive SRS browser, a COCOMO calculator and a Function Point calculator (which use the formulas of Chapter 4), a risk matrix, the DFD viewer, an interactive test runner and software quality metrics based on the McCall and HP FURPS models. Building it ensured that every artefact in this report is backed by data in the application.

5.10 Deployment

The project is built with npm run build (tsc -b && vite build) and deployed to Vercel from the GitHub repository, so each push produces a new build. Supabase credentials are supplied as environment variables; the schema.sql file creates all tables, triggers, policies and seed data in one run in the Supabase SQL editor.

CHAPTER 6
TESTING AND RESULTS

6.1 Test Strategy

Testing combined **black-box** techniques (equivalence partitioning and boundary value analysis), used for the billing and coupon logic where inputs and expected outputs are clear, and **white-box** techniques (statement and branch coverage) used for the order state machine and the delivery logic. The white-box cases were carried out by walking through the code paths, and user-interface behaviour was verified by running the application in the browser for each role and checking it against the functional requirements.

6.2 Black-Box Test Cases

Table 6.1: Black-box test cases for billing and coupons

ID	Req.	Test case	Input	Expected	Actual	Result
TC-01	FR-07	Valid coupon above threshold	SE2026, subtotal ₹550	Discount ₹100 (50% capped)	Discount ₹100	Pass
TC-02	FR-07	Coupon below minimum	SE2026, subtotal ₹200	Error: minimum subtotal ₹250	Error shown, no discount	Pass
TC-03	FR-07	Unknown coupon	ABC123	Error: invalid coupon code	Error shown	Pass
TC-04	FR-07	Percentage cap	CBITFOOD, ₹1000	Discount ₹150 (20% = 200, capped)	Discount ₹150	Pass
TC-05	FR-07	Flat coupon	FIRST50, ₹300	Discount ₹50	Discount ₹50	Pass
TC-06	FR-06	Delivery fee boundary	Subtotal ₹599	Fee ₹35 below 600, free from ₹600	Fee ₹35	Pass
TC-07	FR-06	Delivery fee boundary	Subtotal ₹600	Fee ₹35 below 600, free from ₹600	Fee ₹0	Pass
TC-08	FR-06	Delivery fee boundary	Subtotal ₹601	Fee ₹35 below 600, free from ₹600	Fee ₹0	Pass
TC-09	FR-06	GST rounding	Subtotal ₹655	5% = 32.75 → ₹33	GST ₹33	Pass
TC-10	FR-06	Full bill with coupon	₹655 + SE2026	655 + 0 + 10 + 33 − 100 = ₹598	Total ₹598	Pass
TC-11	FR-05	Empty cart	No items	All fees ₹0, total ₹0	Total ₹0	Pass

The boundary tests TC-06 to TC-08 check the values just below, at and just above the free-delivery threshold, which is where off-by-one errors in the comparison operator would appear. The numbers in the Actual column were produced by evaluating the same rules as the billing code for each input.

6.3 White-Box Test Cases

Table 6.2: White-box test cases

ID	Technique	Unit	Path / condition exercised	Result
WB-01	Statement coverage	Order state machine	PLACED → CONFIRMED → PREPARING → OUT_FOR_DELIVERY → DELIVERED	Pass

ID	Technique	Unit	Path / condition exercised	Result
WB-02	Branch coverage	Merchant action buttons	Each status renders only its own next action (PLACED, CONFIRMED, PREPARING) and none for others	Pass
WB-03	Branch coverage	Delivery fee	subtotal = 0, 0 < subtotal < 600, subtotal ≥ 600	Pass
WB-04	Branch coverage	Coupon engine	Each of SE2026, CBITFOOD, FIRST50 and no coupon; min-subtotal failure branch	Pass
WB-05	Branch coverage	placeOrder()	Supabase not configured; insert succeeds; insert throws	Pass

The billing block has roughly six decision points (two for the delivery fee and platform fee, three coupon branches and the minimum-subtotal check), so its cyclomatic complexity is about seven and at least that many independent paths must be covered; the test set above exercises each coupon branch, the fee boundaries and the empty cart.

6.4 Functional (UI) Test Cases

Table 6.3: Functional test cases per role

ID	Req.	Scenario	Expected behaviour	Result
FT-01	FR-01	Demo login as customer, merchant and driver	Role-specific portal opens	Pass
FT-02	FR-03	Open customer home	Restaurant cards, categories and search shown	Pass
FT-03	FR-04	Select Jumbo, raita and egg add-ons on the mutton biryani	Price updates to ₹600	Pass
FT-04	FR-06/07	Open cart and apply SE2026	Bill shows promo −₹100 and ₹598 payable	Pass
FT-05	FR-08/09	Place order	Tracker opens at “Order Placed” with map markers	Pass
FT-06	FR-10	Open My Orders	Active and past orders listed; reorder available	Pass
FT-07	FR-11/12	Open merchant dashboard	Order queue, menu manager and analytics tabs	Pass
FT-08	FR-13/14	Open driver portal	Earnings, active mission and swipe-to-deliver	Pass
FT-09	FR-15	Toggle theme and reload	Choice persists (localStorage key foodpulse_theme)	Pass

6.5 Quality Attributes

The SE Studio scores software quality against the McCall and HP FURPS models. Our assessment, with the method used for each attribute, is summarised below. It is a self-assessment and not an audited measurement.

Table 6.4: Quality attribute assessment

Attribute	Model	How assessed	Observation
Maintainability	McCall	Separation of components, services, types	Typed modules; services isolate the back end

Attribute	Model	How assessed	Observation
Reliability	FURPS	Fallback to demo mode, try/catch around inserts	App does not crash without back end
Usability	FURPS	Walkthrough of the six-step order flow	Order flow is short and linear
Efficiency	McCall	Static front end, client-side billing	No server round trip for price changes
Security	FURPS	RLS review per table	All eight tables protected
Portability	McCall	Runs in standard browsers	No plug-ins required

6.6 Results: Application Screenshots

The following figures show the running application. Restaurant and dish photographs were replaced by plain placeholders in the screenshots because the image hosts are not reachable from the capture environment; the deployed application shows the photographs.

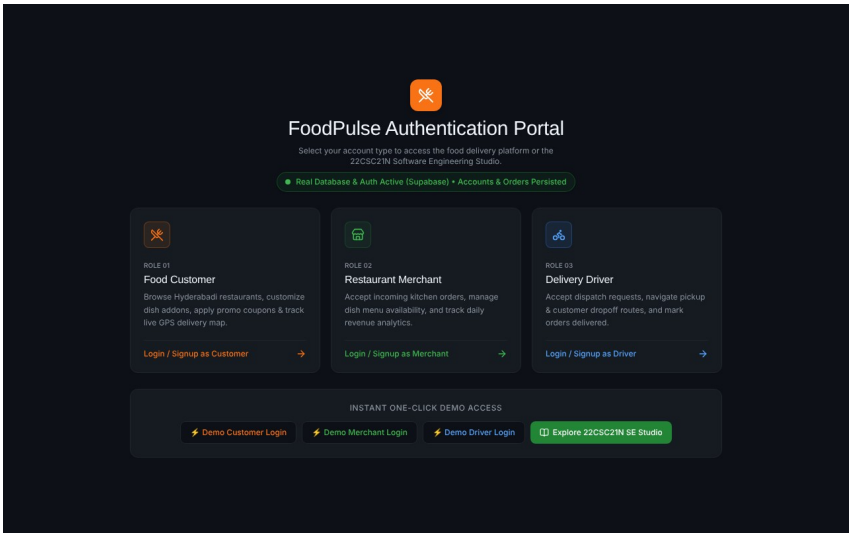


Figure 6.1: Authentication portal with role cards and demo login

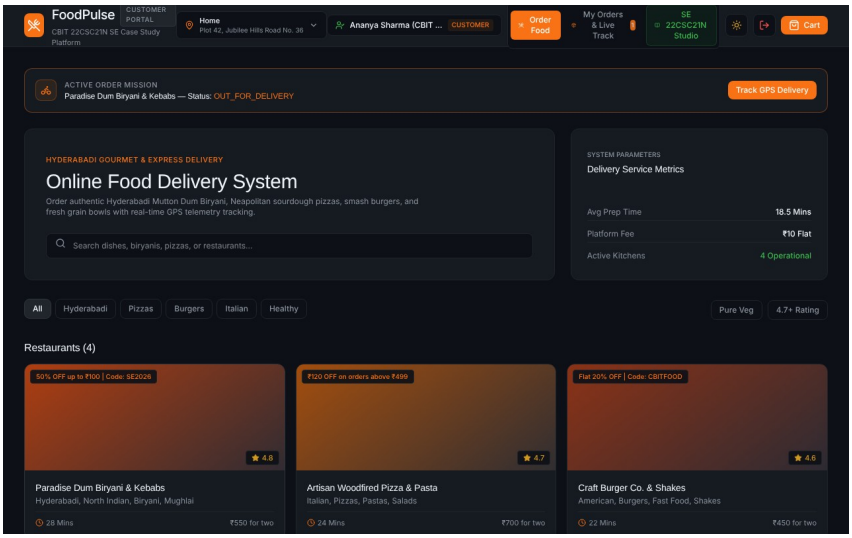


Figure 6.2: Customer home with restaurants, categories and search

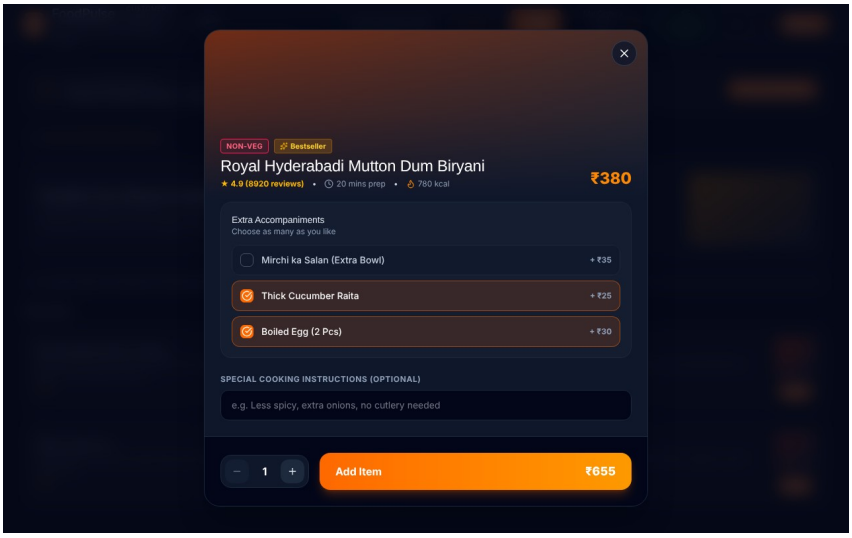


Figure 6.3: Dish modal with size and add-on selection

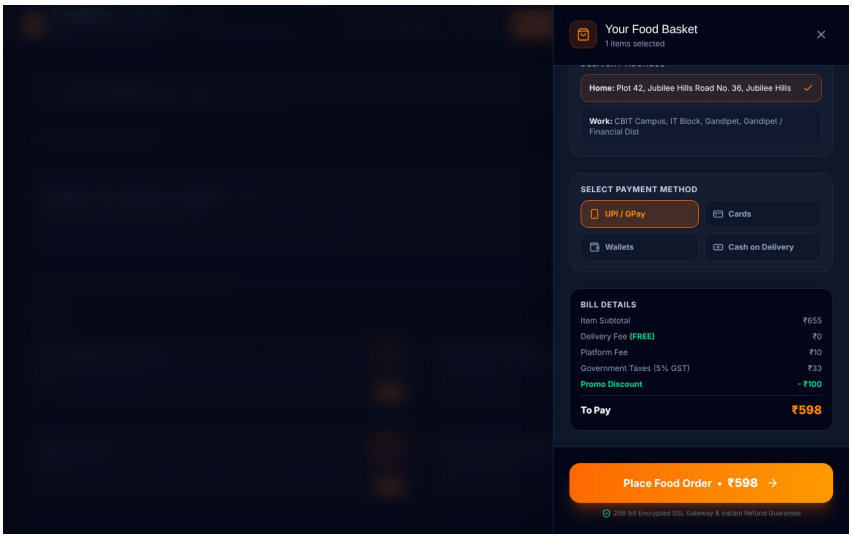


Figure 6.4: Cart drawer with coupon SE2026 applied and bill breakdown

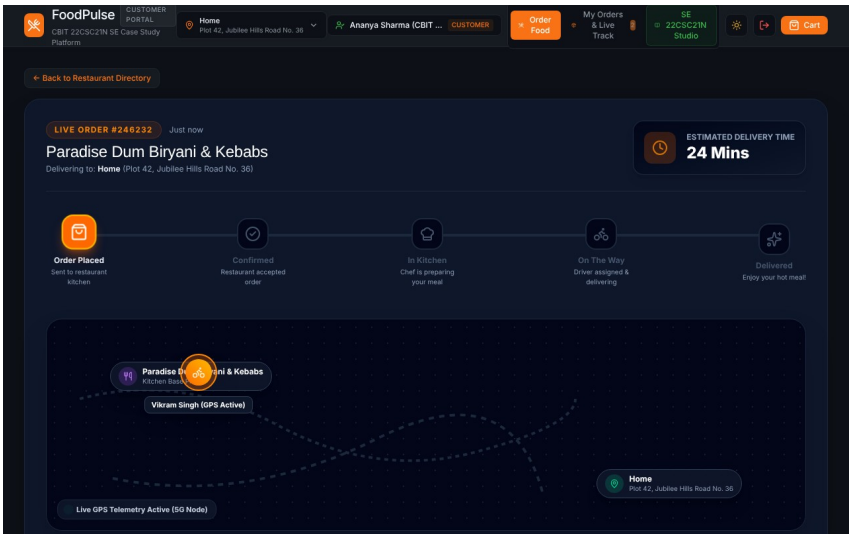


Figure 6.5: Live order tracker with progress stages and map markers

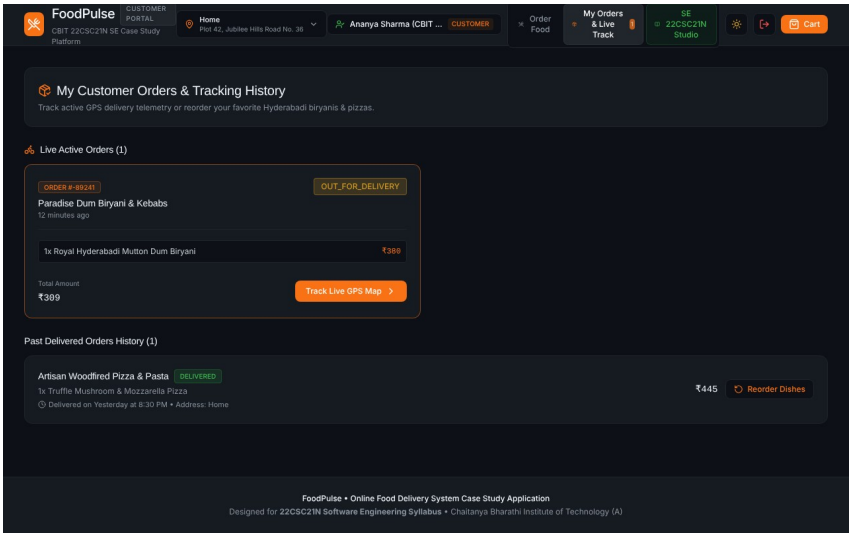


Figure 6.6: My Orders with an active and a delivered order

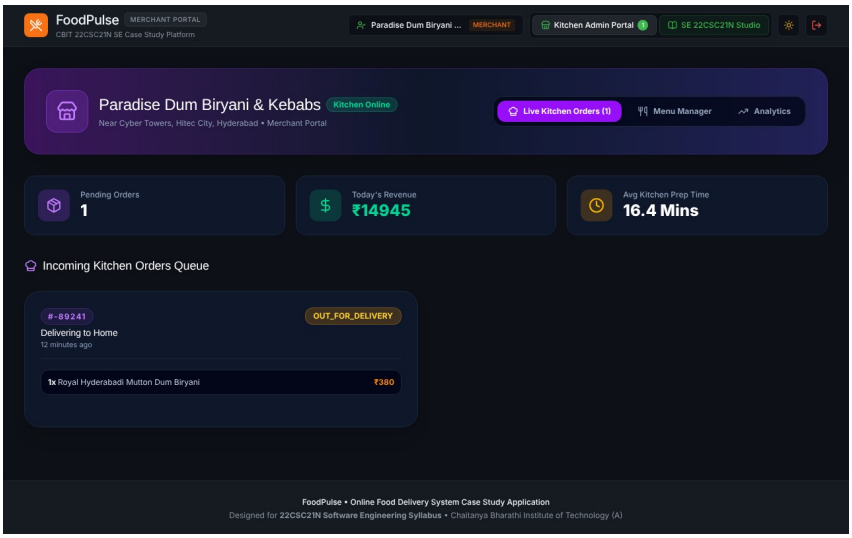


Figure 6.7: Merchant dashboard with live kitchen orders

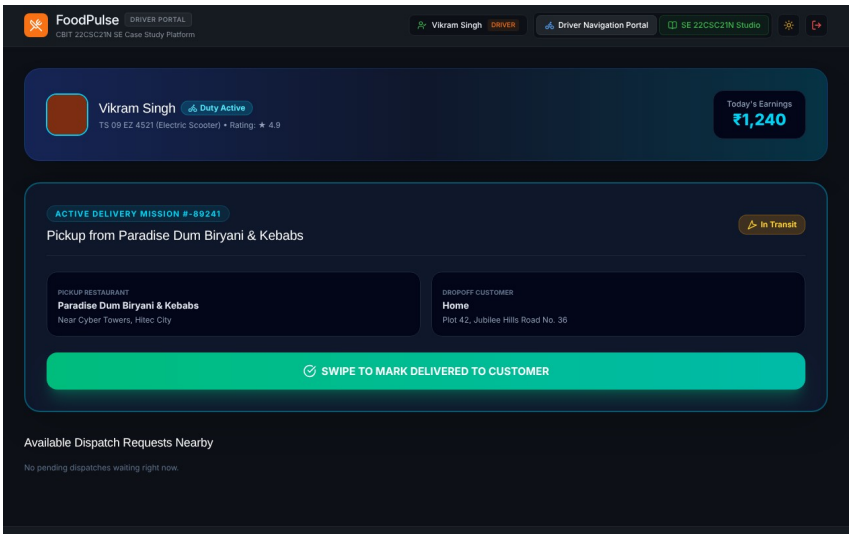


Figure 6.8: Driver portal with active delivery mission

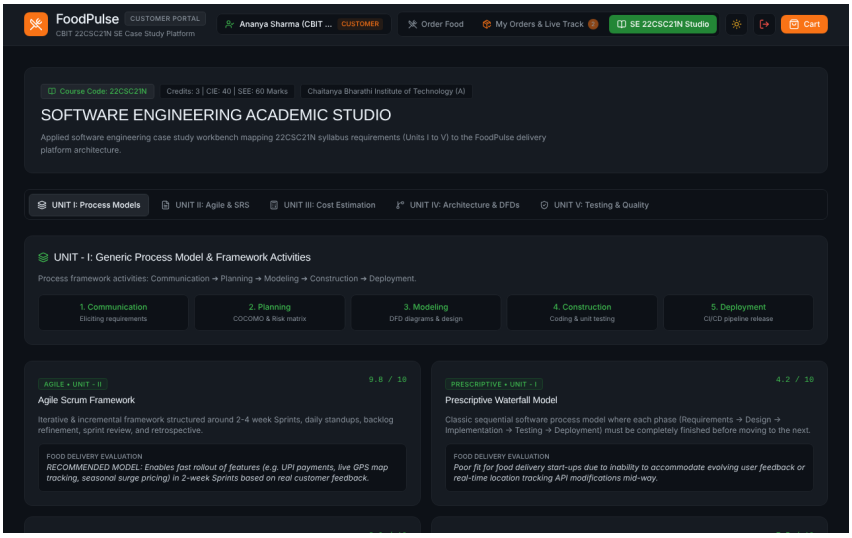


Figure 6.9: SE Studio module showing process models

6.7 Discussion of Results

All 11 black-box, 5 white-box and 9 functional cases passed. The worked example in TC-10 matches the cart screenshot in Figure 6.4 exactly (subtotal ₹655, platform fee ₹10, GST ₹33, promo −₹100, total ₹598), which confirms that the displayed bill and the specified rules agree. The boundary cases showed that the free-delivery threshold behaves as specified at ₹599, ₹600 and ₹601. The main observation from testing is that the order flow is stable across the three roles, while the live tracker (ETA countdown and driver chat) and the payment step are simulations (see Section 7.2).

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This case study applied the Software Engineering process to a realistic product, the FoodPulse Online Food Delivery System. Requirements were specified in IEEE 830 style, the system was modelled with use case, class, sequence, state, DFD and ER diagrams, the effort was estimated with Function Points and COCOMO, risks were registered with mitigation, and the product was implemented with React, TypeScript and Supabase using Agile Scrum. Testing with black-box, white-box and functional cases confirmed that the billing, coupon and order workflows behave as specified.

The most important lessons were practical. Writing the SRS and the state model first made the implementation of the order flow almost mechanical; putting authorisation in Row Level Security kept the client code simple; and isolating all back-end calls in services let the application keep working in a demo mode, which proved valuable for demonstrations. The project also showed that estimation models describe commercial-scale effort, and that reuse of managed services and libraries can reduce actual effort far below that estimate.

7.2 Limitations

- Payment is simulated: no real payment gateway is integrated, so no money moves.
- The live map and the ETA countdown are simulated; real GPS coordinates are not yet streamed from a driver's device.
- Order placement in database mode currently uses a single seeded restaurant identifier; multi-restaurant carts are not yet split into separate orders.
- Driver assignment is manual (the driver accepts); there is no automatic matching by distance.
- There is no automated test suite in the repository; the tests in Chapter 6 were executed manually and by evaluating the billing rules.

7.3 Future Scope

- Integrate a payment gateway such as Razorpay with server-side verification and webhooks.
- Stream real driver locations with the browser Geolocation API into driver_locations and render them on an interactive map.
- Add automatic driver matching using distance and load, and surge pricing for peak hours.
- Introduce push notifications and SMS for order updates, and package the app as a Progressive Web App.
- Add an administrator dashboard, coupon management and restaurant onboarding.
- Write automated unit tests (Vitest) and end-to-end tests (Playwright) and run them in continuous integration.

BIBLIOGRAPHY

[1] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed., McGraw-Hill Education, 2019.

[2] I. Sommerville, Software Engineering, 10th ed., Pearson, 2015.

[3] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE, 1998.

[4] G. Booch, J. Rumbaugh and I. Jacobson, The Unified Modeling Language User Guide, 2nd ed., Addison-Wesley, 2005.

[5] B. W. Boehm, Software Engineering Economics, Prentice Hall, 1981.

[6] A. J. Albrecht, "Measuring application development productivity," Proc. IBM Applications Development Symposium, 1979.

[7] K. Schwaber and J. Sutherland, The Scrum Guide, 2020. https://scrumguides.org

[8] G. J. Myers, C. Sandler and T. Badgett, The Art of Software Testing, 3rd ed., Wiley, 2011.

[9] J. A. McCall, P. K. Richards and G. F. Walters, Factors in Software Quality, RADC-TR-77-369, 1977.

[10] React Documentation. https://react.dev

[11] Supabase Documentation (Auth, Database, Row Level Security, Realtime). https://supabase.com/docs

[12] PostgreSQL Documentation, Row Security Policies. https://www.postgresql.org/docs/

[13] Vite Documentation. https://vite.dev | Tailwind CSS Documentation. https://tailwindcss.com/docs

[14] FoodPulse source code. https://github.com/vamshi1010-cr/food-pulse-delivery

APPENDIX

Appendix A: Database Schema Excerpt (supabase/schema.sql)

```
CREATE TABLE IF NOT EXISTS public.orders (
  id            UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  customer_id   UUID REFERENCES public.profiles(id),
  restaurant_id UUID REFERENCES public.restaurants(id),
  driver_id     UUID REFERENCES public.profiles(id),
  status        TEXT DEFAULT 'PLACED'
     CHECK (status IN ('PLACED','CONFIRMED','PREPARING','READY',
                       'OUT_FOR_DELIVERY','DELIVERED','CANCELLED')),
  subtotal      NUMERIC(10,2) NOT NULL,
  delivery_fee  NUMERIC(10,2) DEFAULT 0,
  platform_fee  NUMERIC(10,2) DEFAULT 10,
  tax           NUMERIC(10,2) DEFAULT 0,
  discount      NUMERIC(10,2) DEFAULT 0,
  total_amount  NUMERIC(10,2) NOT NULL,
  payment_method TEXT DEFAULT 'UPI',
  created_at    TIMESTAMPTZ DEFAULT NOW()
);
```

```
ALTER TABLE public.orders ENABLE ROW LEVEL SECURITY;
```

Appendix B: Project Links

- GitHub repository: https://github.com/vamshi1010-cr/food-pulse-delivery
- Deployed application: https://fleet-maple-g6u25hp-q1j5t99gs-vamshi1010-crs-projects.vercel.app